

Types de données simples

Nombres, chaînes de caractères, booléens — premiers pas en Pyret

Traduit et adapté depuis Bootstrap World (Fall 2026)

Licence Creative Commons 4.0 International

Table des matières

1	Chaînes de caractères et nombres	3
1.1	Lancement	3
1.2	Exploration	4
1.2.1	Nombres	5
1.2.2	Chaînes de caractères	5
1.2.3	Messages d'erreur	6
1.2.4	Pièges fréquents	6
1.3	Synthèse	7
2	Booléens	8
2.1	Lancement	8
2.2	Exploration	9
2.2.1	Types et comparaisons	9
2.3	Synthèse	10

Chaînes de caractères et nombres

Objectif de la section

En travaillant ensemble selon le modèle Conducteur/Navigateur, les élèves expérimentent avec l'éditeur Pyret. Ils explorent les types de données Nombre et Chaîne de caractères, et leur comportement dans Pyret.

Lancement

Aujourd'hui, nous allons commencer à programmer en Pyret !



FIGURE 1 – Icône Conducteur/Navigateur

Quand vous programmez dans ce cours, vous travaillerez en binôme selon le modèle **Conducteur/Navigateur** (*Driver/Navigator*).

- Chaque groupe ne peut avoir qu'un seul **Conducteur** — ses mains sont sur le clavier, son travail consiste à gérer la frappe, les clics, les raccourcis, etc.
- Si vous n'êtes pas Conducteur, vous êtes **Navigateur** — votre travail est de dire au Conducteur où aller, quoi taper, etc.
- Un bon Conducteur ne tape *que* ce que le Navigateur lui dit de taper, et un bon Navigateur donne des instructions claires et précises.

Notes pour l'enseignant·e

Ce modèle de programmation en binôme est extrêmement utile pour séparer l'étape de « réflexion » de celle de « frappe ». Les élèves — surtout ceux qui découvrent la programmation textuelle ou la frappe elle-même — peuvent avoir du mal à mettre leurs idées dans l'environnement de programmation.

Ce modèle leur permet de se concentrer sur la *communication de leurs idées*, en laissant le Conducteur se concentrer sur le code. De même, le Conducteur a l'occasion de se concentrer sur la syntaxe et la programmation. Enfin, l'obligation de faire passer les idées par les mains d'une autre personne est un excellent échafaudage pour amener les élèves à parler de leur raisonnement et du code.

Activité

- Ouvrez pyret.BootstrapWorld.org (BPO).
- Qu'est-ce que vous remarquez ? Qu'est-ce qui vous intrigue ?

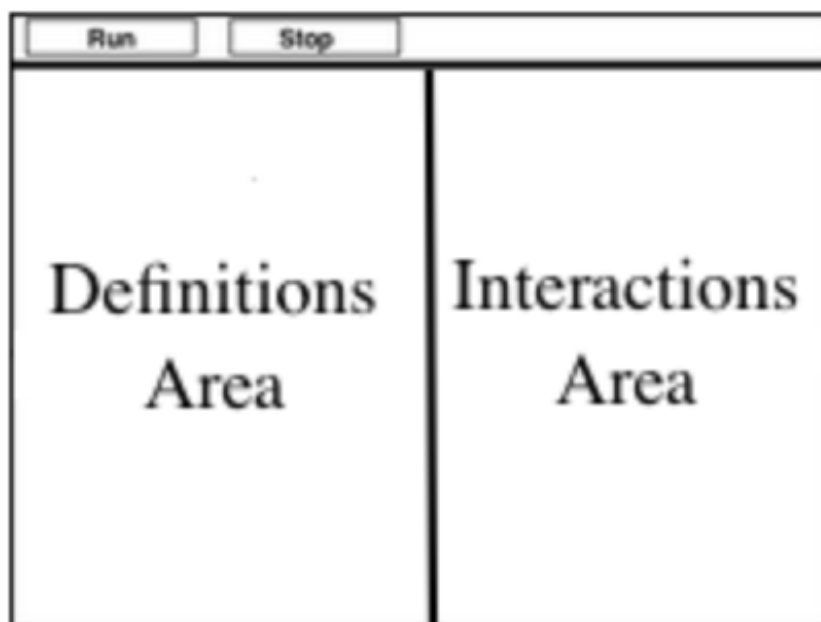


FIGURE 2 – L’environnement d’édition Pyret : les définitions à gauche, les interactions à droite

Cet écran s’appelle l’**éditeur**. Il y a quelques boutons en haut, mais la majeure partie de l’écran est occupée par deux grandes zones : la **Zone de Définitions** à gauche et la **Zone d’Interactions** à droite.

- La **Zone de Définitions** est l’endroit où les programmeurs définissent des valeurs et des fonctions qu’ils veulent conserver.
- La **Zone d’Interactions** leur permet d’expérimenter avec ces valeurs et fonctions. C’est comme si on mettait un ensemble de définitions de fonctions au tableau et qu’on demandait aux élèves d’utiliser ces fonctions pour calculer des réponses sur du papier brouillon.

Point clé

Chaque fois qu’on ouvre un fichier ou qu’on fait une modification dans la Zone de Définitions, il faut cliquer sur « **Run** » pour charger le programme. Cliquer sur « Run » efface également la Zone d’Interactions. Pour l’instant, nous écrivons uniquement des programmes dans la Zone d’Interactions à droite.

Exploration

Les mathématiques sont un langage, tout comme le français, l’anglais ou n’importe quelle autre langue.

- Les langues ont des **noms** (ex. : « ballon », « tomate », etc.) et des **verbes**, qui sont des actions qu'on peut effectuer sur ces noms (ex. : je peux « lancer un ballon »).
 - Les maths et la programmation ont des **valeurs**, comme les nombres 1, 2 et 3.
 - Au lieu de verbes, les maths et la programmation ont des **fonctions**, qui sont des actions qu'on peut effectuer sur des valeurs (ex. : « je peux élever un nombre au carré »).
- Les langues ont des règles de **syntaxe**.
 - En français, par exemple, les mots n'ont pas de ! et ? au milieu.
 - En maths et en programmation, les nombres n'ont pas de & dedans.
- Les langues ont aussi des règles de **grammaire**.
 - *Le chat dort.* est une phrase.
 - *Dort le chat.* est grammaticalement bizarre, même si tous les mots ont une syntaxe valide.
 - L'ordre des mots compte !

Point clé

Garder à l'esprit l'importance de la **syntaxe** et de la **grammaire** est utile quand on apprend à programmer !

Nombres

Essayons quelques expressions simples dans la Zone d'Interactions :

Code

```
– 1      → 1
– -4.7    → -4.7
– 1/3     → 1/3 (cliquez pour voir le nombre décimal 0.333333)
– 6 * 4    → 24
– 2 + 3 * 4 → 14 (la multiplication est prioritaire)
– (2 + 3) * 4 → 20 (les parenthèses changent l'ordre)
– 10 - 3 + 1 → 8
```

Chaînes de caractères

Une **chaîne de caractères** (*String*) est tout ce qui est entre guillemets :

Code

```
– "Bonjour, monde !" → "Bonjour, monde !"
– "42" → "42" (c'est une chaîne, pas un nombre!)
– "hello" + "world" → "helloworld"
```

Messages d'erreur

Activité

- Avez-vous obtenu des messages d'erreur ? Qu'en avez-vous appris ?

Notes pour l'enseignant·e

La plupart des messages d'erreur que nous avons vus attirent notre attention sur des erreurs de syntaxe : virgules manquantes, guillemets non fermés, etc.

Les messages d'erreur sont un moyen pour Pyret d'expliquer ce qui n'a pas fonctionné, et sont vraiment utiles pour trouver les erreurs. Soulignez leur utilité et expliquez pourquoi les élèves devraient *lire ces messages à voix haute* avant de demander de l'aide.

Faites découvrir aux élèves les erreurs suivantes :

Erreur

- $6 / 0$ Comme les humains, Pyret ne peut pas diviser par zéro et donne une erreur.
- $(2 + 2$ Une parenthèse non fermée pose problème, tout comme un guillemet non fermé.

Pièges fréquents

Notes pour l'enseignant·e

En Pyret, écrire un nombre décimal comme `.5` (sans le zéro devant) produit une erreur de syntaxe. Assurez-vous que les élèves comprennent que Pyret exige que les nombres décimaux commencent par un zéro : `0.5` et non `.5` !

Synthèse

Réflexion

- **Qu'avons-nous appris sur Pyret?**
 - Les Nombres et les Chaînes s'évaluent en eux-mêmes.
 - Notre éditeur est assez malin et peut basculer automatiquement entre l'affichage d'un nombre rationnel en fraction ou en décimal, simplement en cliquant dessus!
 - Tout ce qui est entre guillemets est une Chaîne, même quelque chose comme "42".
 - Les Chaînes *doivent* avoir des guillemets des deux côtés.
 - Les opérateurs comme +, -, * et / ont besoin d'espaces autour d'eux.
 - Dans Pyret, les opérateurs fonctionnent exactement comme en mathématiques.
 - Dès qu'il y a plus d'un opérateur, Pyret exige l'utilisation de parenthèses pour définir l'ordre des opérations.
 - **Les types comptent!** On peut additionner deux Nombres ou deux Chaînes entre eux, mais on ne peut pas additionner le Nombre 4 à la Chaîne "bonjour".
- Quelles autres questions avez-vous sur le fonctionnement des Chaînes et des Nombres dans Pyret?

Booléens

Objectif de la section

Cette leçon présente aux élèves les Booléens, un type de données unique avec seulement deux valeurs : `true` et `false`, et explique pourquoi ils sont utiles à la fois dans le monde réel et dans l'environnement de programmation.

Lancement

Réflexion

Quelle est la réponse : est-ce que 3 est plus grand que 10 ?

Les expressions qui produisent des Booléens sont des questions oui-ou-non et s'évaluent toujours soit à `true` (« oui »), soit à `false` (« non »).

Point clé

La capacité de séparer des entrées en **deux catégories** est unique et très utile !

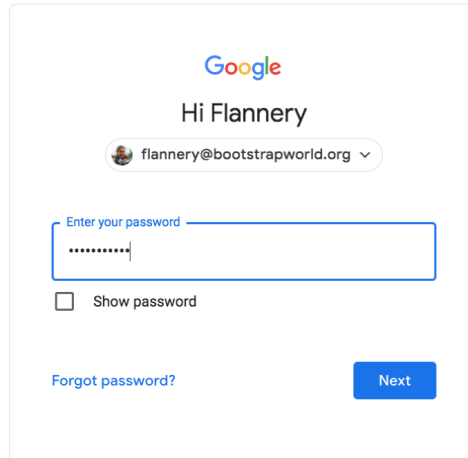
A screenshot of a Google login interface. At the top is the Google logo. Below it, the text "Hi Flannery" is displayed. Underneath is a profile picture placeholder and the email address "flannery@bootstrapworld.org" with a dropdown arrow. A password input field is shown with the placeholder text "Enter your password" and a series of dots for the password. Below the password field is a checkbox labeled "Show password". At the bottom left is a link "Forgot password?" and at the bottom right is a blue button labeled "Next".

FIGURE 3 – Panneau de taille minimum pour un manège

Par exemple : certains montagnes russes avec des loopings exigent une taille minimum pour s'assurer que les passagers sont bien maintenus par les harnais universels. Le contrôleur ne se soucie pas de votre taille exacte ; il vérifie juste si vous êtes aussi grand que la marque sur le poteau. Si vous êtes assez grand, vous pouvez monter ; sinon, on ne vous laisse pas entrer parce qu'on ne peut pas garantir votre sécurité.

De même, quand vous vous connectez à votre messagerie, l'ordinateur demande votre mot de passe et vérifie s'il correspond à ce qui est enregistré. Si la correspondance est `true`, il vous amène à vos messages, mais si ce que vous entrez ne correspond pas, vous obtenez un message d'erreur.

Activité

Brainstorming : trouvez d'autres situations où les Booléens sont utiles, dans la vie réelle et en programmation.

Exploration

Les élèves font des prédictions sur ce que diverses expressions booléennes vont renvoyer, puis les testent dans l'éditeur.

Code

Voici les opérateurs de comparaison en Pyret :

Opérateur	Signification	Exemple
<	strictement inférieur	3 < 10 → true
>	strictement supérieur	3 > 10 → false
<=	inférieur ou égal	5 <= 5 → true
>=	supérieur ou égal	7 >= 3 → true
==	égal	4 == 4 → true
<>	différent	4 <> 5 → true

Types et comparaisons

Activité

Essayez ces expressions dans Pyret et notez les résultats :

- 10 > 5
- 10 < 5
- 10 == 10
- 10 <> 10
- "chat" == "chat"
- "chat" == "chien"
- 10 > "5" (que se passe-t-il?)

Point clé

On ne peut pas comparer un Nombre avec une Chaîne ! Pyret donne une erreur parce que les **types** ne sont pas compatibles.

Le type détermine ce qu'on peut faire avec une valeur :

- On peut additionner deux Nombres : $3 + 4 \rightarrow 7$
- On peut concaténer deux Chaînes : `"bon" + "jour" → "bonjour"`
- On peut comparer deux Nombres : $5 > 3 \rightarrow \text{true}$
- Mais on ne peut pas additionner un Nombre et une Chaîne : $3 + \text{"jour"} \rightarrow \text{erreur!}$

Synthèse

Réflexion

- **Qu'est-ce qui distingue les Booléens des autres types de données ?**
 - Ils permettent de séparer les données en deux tas : les valeurs qui renvoient `true` et celles qui renvoient `false`.
- **Pourquoi est-ce utile ?**
 - Cela permet de prendre des **décisions** dans un programme.
 - C'est la base des **conditions** (si/alors/sinon) qu'on verra plus tard.

Résumé : les trois types de données simples

Type	Exemples	Opérations	Valeurs possibles
Nombre	42, -3.14, 1/3	+, -, *, /	Une infinité
Chaîne	"bonjour", "42"	+ (concaténation)	Une infinité
Booléen	true, false	<, >, ==, <>	Seulement 2 : true ou false

Point clé

Les types sont votre ami !

Quand une erreur de type survient, Pyret vous le dit. C'est une bonne chose : cela vous évite d'écrire du code qui n'a pas de sens. Apprendre à lire les messages d'erreur de type est une compétence fondamentale en programmation.

À propos de ces supports

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).



Bootstrap par la Communauté Bootstrap est sous licence Creative Commons 4.0 Unported.

Traduction française réalisée en juillet 2026.

Source originale : <https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/simple-data-types/>